

Formal Methods

Giovanni Raserà

What's wrong with this code ?

```
24 // sqrt((p1.x - p2.x)^2 + (p1.y - p2.y)^2)
25 float distance(Point p1, Point p2){
26     float x_diff = p1.x - p2.x;
27     float y_diff = p1.y - p2.y;
28
29     float mid = std::pow(x_diff, 2) - std::pow(y_diff, 2);
30
31     return std::pow(mid, 2);
32 }
```

What's wrong with this code ?

```
24 // sqrt((p1.x - p2.x)^2 + (p1.y - p2.y)^2)
25 float distance(Point p1, Point p2){
26     float x_diff = p1.x - p2.x;
27     float y_diff = p1.y - p2.y;
28
29     float mid = std::pow(x_diff, 2) - std::pow(y_diff, 2);
30
31     return std::pow(mid, 2);
32 }
```


Why does this matter ?

Ariane 5 explosion

Why does this matter ?

Ariane 5 explosion

What ?

- The **halt** of the **inertial navigation system** led to the **explosion**

Why ?

- Dead code with **numeric operation exception that was handles inappropriately**

Why does this matter ?

Ariane 5 explosion

What ?

- The **halt** of the **inertial navigation system** led to the **explosion**

Why ?

- Dead code with **numeric operation exception** that was **handles inappropriately**

```
// Single use
int explodeRocket(int m, int s){
    int more = m + s;

    return m;
}
```

Why does this matter ?

Ariane 5 explosion

What ?

- The **halt** of the **inertial navigation system** led to the **explosion**

Why ?

- Dead code with **numeric operation exception** that was handles inappropriately

```
// Single use
int explodeRocket(int m, int s){
    int more = m + s;

    return m;
}
```



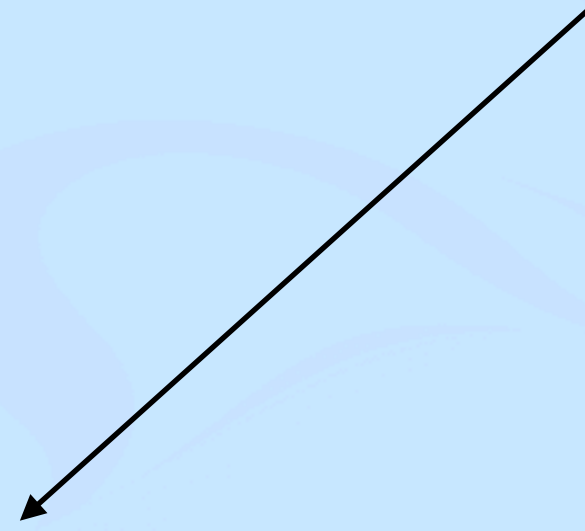
What can we do ?

What can we do ?

We can use formal methods

What can we do ?

We can use **formal methods**

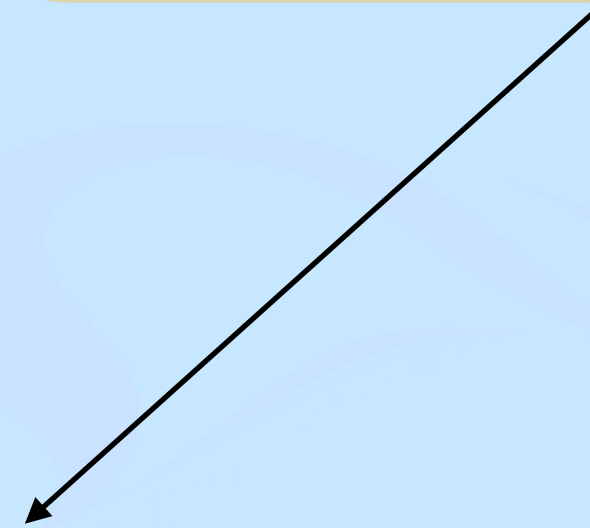


Mathematical methods to prove:

- Correctness
- Termination
- Deadlocks Free

What can we do ?

We can use **formal methods**



Mathematical methods to prove:

- Correctness
- Termination
- Deadlocks Free

Under fair scheduling assumption



Correctness

Is this program correct ?

```
24 // sqrt((p1.x - p2.x)^2 + (p1.y - p2.y)^2)
25 float distance(Point p1, Point p2){
26     float x_diff = p1.x - p2.x;
27     float y_diff = p1.y - p2.y;
28
29     float mid = std::pow(x_diff, 2) - std::pow(y_diff, 2);
30
31     return std::pow(mid, 2);
32 }
```


Correctness

Is this program correct ?

```
24 // sqrt((p1.x - p2.x)^2 + (p1.y - p2.y)^2)
25 float distance(Point p1, Point p2){
26     float x_diff = p1.x - p2.x;
27     float y_diff = p1.y - p2.y;
28
29     float mid = std::pow(x_diff, 2) - std::pow(y_diff, 2);
30
31     return std::pow(mid, 2);
32 }
```

- NO !
- Wrong Implementation !

Correctness

Is this program correct ?

```
24 // sqrt((p1.x - p2.x)^2 + (p1.y - p2.y)^2)
25 float distance(Point p1, Point p2){
26     float x_diff = p1.x - p2.x;
27     float y_diff = p1.y - p2.y;
28
29     float mid = std::pow(x_diff, 2) - std::pow(y_diff, 2);
30
31     return std::pow(mid, 2);
32 }
```

- NO !
- Wrong Implementation !

```
14 // sqrt((p1.x - p2.x)^2 + (p1.y - p2.y)^2)
15 float distanceCorrect(Point p1, Point p2){
16     float x_diff = p1.x - p2.x;
17     float y_diff = p1.y - p2.y;
18
19     float mid = std::pow(x_diff, 2) + std::pow(y_diff, 2);
20
21     return std::sqrt(mid);
22 }
```

Correctness

Is this program correct ?

```
24 // sqrt((p1.x - p2.x)^2 + (p1.y - p2.y)^2)
25 float distance(Point p1, Point p2){
26     float x_diff = p1.x - p2.x;
27     float y_diff = p1.y - p2.y;
28
29     float mid = std::pow(x_diff, 2) - std::pow(y_diff, 2);
30
31     return std::pow(mid, 2);
32 }
```

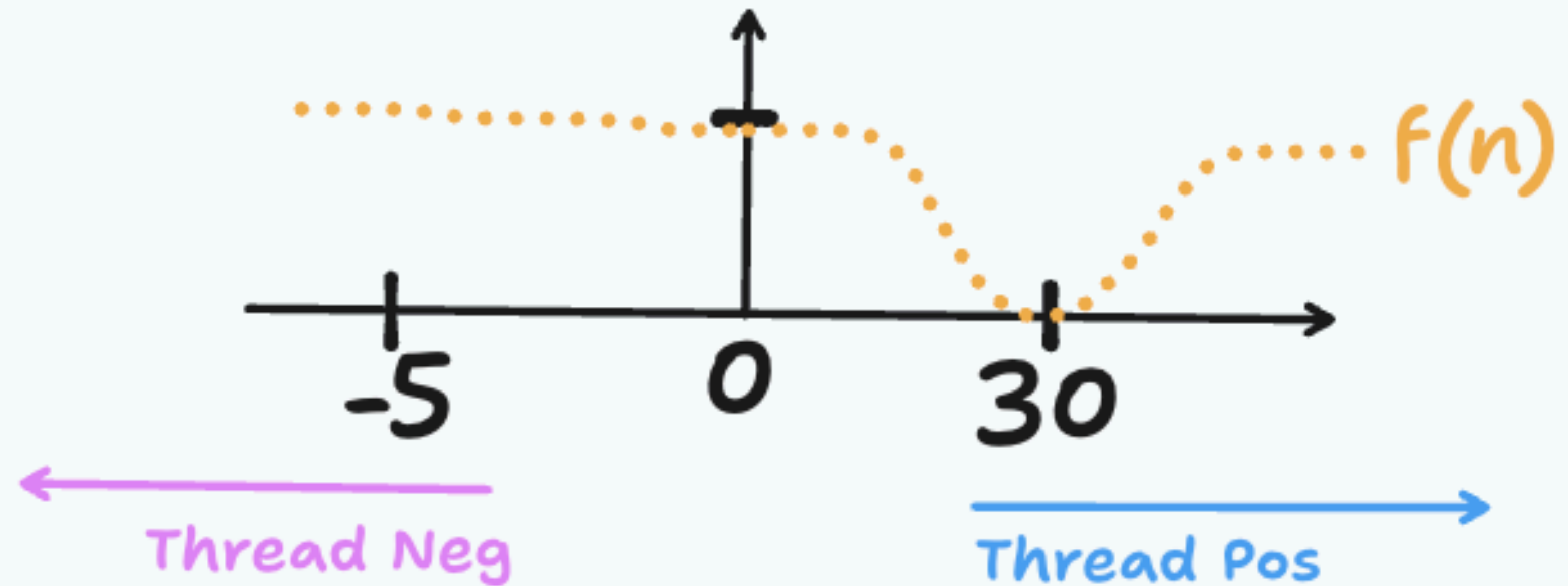
- NO !
- Wrong Implementation !

```
14 // sqrt((p1.x - p2.x)^2 + (p1.y - p2.y)^2)
15 float distanceCorrect(Point p1, Point p2){
16     float x_diff = p1.x - p2.x;
17     float y_diff = p1.y - p2.y;
18
19     float mid = std::pow(x_diff, 2) + std::pow(y_diff, 2);
20
21     return std::sqrt(mid);
22 }
```

- YES !
- We can prove it **using the structure** of the program **annotated with Hoare Triples**.

Termination

Termination



$$f(n) := \begin{cases} 0 & \text{if } n == 30 \\ 1 & \text{otherwise} \end{cases}$$

Termination

Does this program terminate for every possible execution ?

```
5 // Search zero with negative numbers
6 int searchNeg(bool *found, std::function<int(int)> f){
7     int x = 1;
8     while(!(*found)){
9         x--;
10        *found = (f(x) == 0);
11    }
12    return x;
13 }
14
15 // Search zero with positive numbers
16 int searchPos(bool *found, std::function<int(int)> f){
17     int y = 0;
18     while(!(*found)){
19         y++;
20        *found = (f(y) == 0);
21    }
22    return y;
23 }
24
25 int main(){
26     auto myF = [](int x){return x == 30 ? 0 : 1;};
27     bool found = false;
28     auto fNeg = std::async(std::launch::async, searchNeg, &found, myF);
29     auto fPos = std::async(std::launch::async, searchPos, &found, myF);
30     auto resNeg = fNeg.get();
31     auto resPos = fPos.get();
32 }
```

Termination

Does this program terminate for every possible execution ?

```
5 // Search zero with negative numbers
6 int searchNeg(bool *found, std::function<int(int)> f){
7     int x = 1;
8     while(!(*found)){
9         x--;
10        *found = (f(x) == 0);
11    }
12    return x;
13 }
14
15 // Search zero with positive numbers
16 int searchPos(bool *found, std::function<int(int)> f){
17     int y = 0;
18     while(!(*found)){
19         y++;
20        *found = (f(y) == 0);
21    }
22    return y;
23 }
24
25 int main(){
26     auto myF = [](int x){return x == 30 ? 0 : 1;};
27     bool found = false;
28     auto fNeg = std::async(std::launch::async, searchNeg, &found, myF);
29     auto fPos = std::async(std::launch::async, searchPos, &found, myF);
30     auto resNeg = fNeg.get();
31     auto resPos = fPos.get();
32 }
```

- NO !
- **Interference**
 - Line 10 vs Line 20

Termination

Does this program terminate for every possible execution ?

```
5 // Search zero with negative numbers
6 int searchNeg(bool *found, std::function<int(int)> f){
7     int x = 1;
8     while(!(*found)){
9         x--;
10        *found = (f(x) == 0);
11    }
12    return x;
13 }
14
15 // Search zero with positive numbers
16 int searchPos(bool *found, std::function<int(int)> f){
17     int y = 0;
18     while(!(*found)){
19         y++;
20        *found = (f(y) == 0);
21    }
22    return y;
23 }
24
25 int main(){
26     auto myF = [](int x){return x == 30 ? 0 : 1;};
27     bool found = false;
28     auto fNeg = std::async(std::launch::async, searchNeg, &found, myF);
29     auto fPos = std::async(std::launch::async, searchPos, &found, myF);
30     auto resNeg = fNeg.get();
31     auto resPos = fPos.get();
32 }
```

- NO !

Interference

- Line 10 vs Line 20

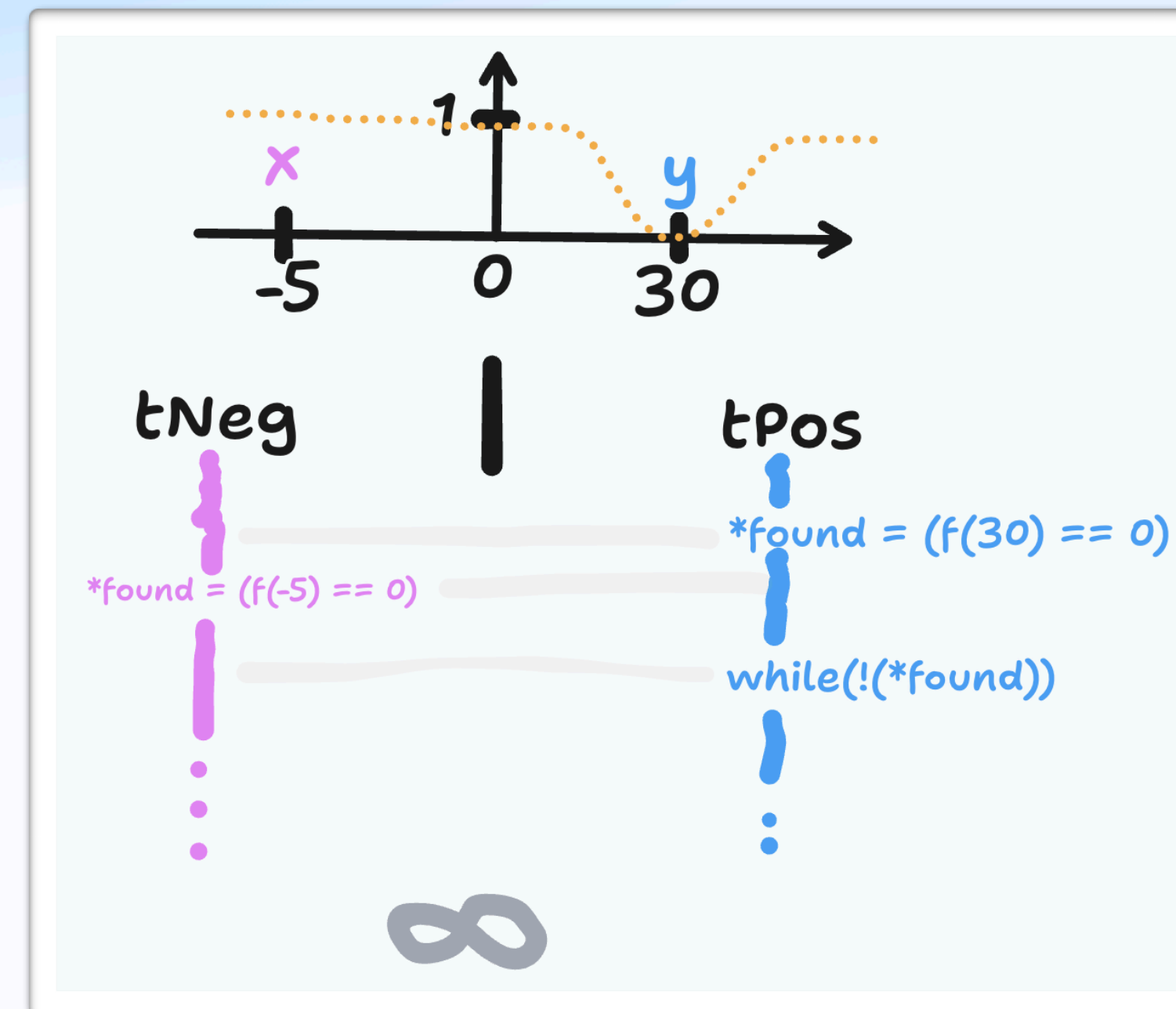
Termination

Does this program terminate for every possible execution ?

```
5 // Search zero with negative numbers
6 int searchNeg(bool *found, std::function<int(int)> f){
7     int x = 1;
8     while(!(*found)){
9         x--;
10        *found = (f(x) == 0);
11    }
12    return x;
13 }
14
15 // Search zero with positive numbers
16 int searchPos(bool *found, std::function<int(int)> f){
17     int y = 0;
18     while(!(*found)){
19         y++;
20        *found = (f(y) == 0);
21    }
22    return y;
23 }
24
25 int main(){
26     auto myF = [](int x){return x == 30 ? 0 : 1;};
27     bool found = false;
28     auto fNeg = std::async(std::launch::async, searchNeg, &found, myF);
29     auto fPos = std::async(std::launch::async, searchPos, &found, myF);
30     auto resNeg = fNeg.get();
31     auto resPos = fPos.get();
32 }
```

• NO !

Interference



Deadlock Free

Is this program deadlock free?

```
8  // Search zero with negative numbers
9  int searchNeg(bool *found, std::function<int(int)> f){
10     int x = 1;
11     while(!(*found)){
12         x--;
13         { // Atomic
14             std::lock_guard<std::mutex> lk{mx};
15             if(f(x) == 0) *found = true;
16         }
17     }
18     return x;
19 }
20
21
22 // Search zero with positive numbers
23 int searchPos(bool *found, std::function<int(int)> f){
24     int y = 0;
25     while(!(*found)){
26         y++;
27         { // Atomic
28             std::lock_guard<std::mutex> lk{mx};
29             if(f(y) == 0) *found = true;
30             if(*found) std::cout << "Found: f(" << y << ") = 0" << std::endl;
31         }
32     }
33     return y;
34 }
35
36 int main(){
37     auto myF = [](int x){return x == 30 ? 0 : 1;};
38     bool found = false;
39     auto fNeg = std::async(std::launch::async, searchNeg, &found, myF);
40     auto fPos = std::async(std::launch::async, searchPos, &found, myF);
41     auto resNeg = fNeg.get();
42     auto resPos = fPos.get();
43 }
```


Deadlock Free

Is this program deadlock free?

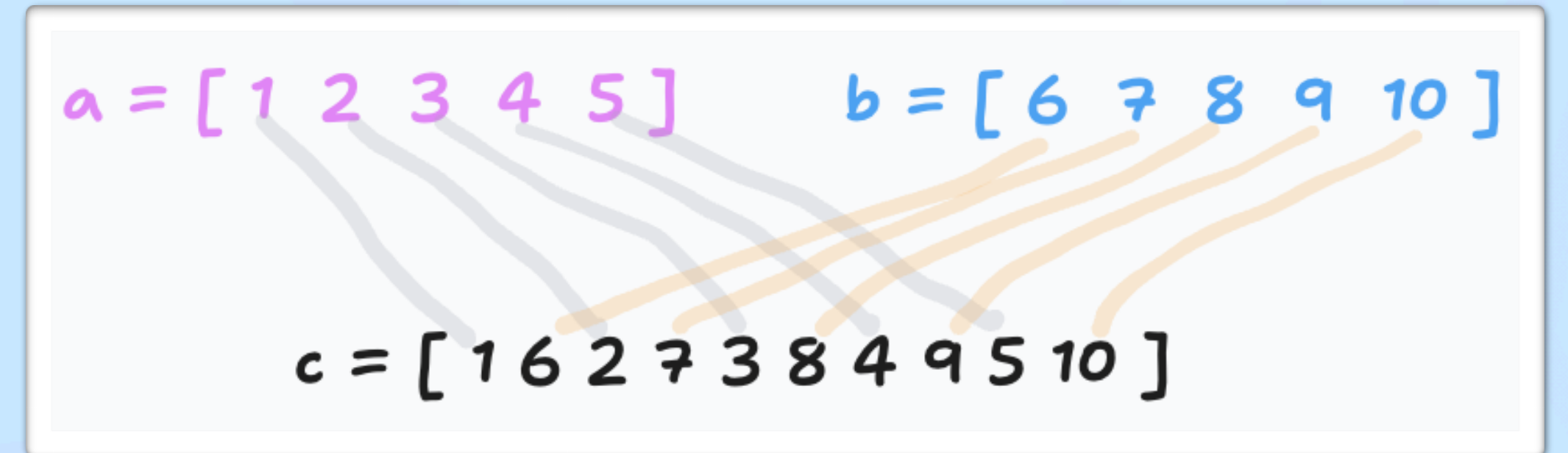
```
8 // Search zero with negative numbers
9 int searchNeg(bool *found, std::function<int(int)> f){
10     int x = 1;
11     while(!(*found)){
12         x--;
13         { // Atomic
14             std::lock_guard<std::mutex> lk{mx};
15             if(f(x) == 0) *found = true;
16         }
17     }
18     return x;
19 }
20
21
22 // Search zero with positive numbers
23 int searchPos(bool *found, std::function<int(int)> f){
24     int y = 0;
25     while(!(*found)){
26         y++;
27         { // Atomic
28             std::lock_guard<std::mutex> lk{mx};
29             if(f(y) == 0) *found = true;
30             if(*found) std::cout << "Found: f(" << y << ") = 0" << std::endl;
31         }
32     }
33     return y;
34 }
35
36 int main(){
37     auto myF = [](int x){return x == 30 ? 0 : 1;};
38     bool found = false;
39     auto fNeg = std::async(std::launch::async, searchNeg, &found, myF);
40     auto fPos = std::async(std::launch::async, searchPos, &found, myF);
41     auto resNeg = fNeg.get();
42     auto resPos = fPos.get();
43 }
```

- YES !
- By checking if all the combinations of the atomic region exclude deadlocks.
- This code is:
 - Totally Correct
 - Deadlock Free

Example

Parallel vector copy

1: write the program in a pseudo language



Thread 1

```
 $S'_1$  = while  $i < M$  do  
    await  $B$  then  $c[k] := a[i]; k := k + 1; B := false; i := i + 1$  end  
od
```

Thread 2

```
 $S'_2$  = while  $j < M$  do  
    await  $\neg B$  then  $c[k] := b[j]; k := k + 1; B := true; j := j + 1$  end  
od
```

Main

```
 $S' = k := 0; B := true; i := 0; j := 0; [S'_1 | S'_2]$ 
```

2: Annotate the program with a Proof Outline

Thread 1

$$\begin{array}{l} \{inv : I_1\} \{bd : t_1\} \\ \textbf{while } i < M \textbf{ do } \{I_1 \wedge i < M\} \\ \quad \textbf{await } B \textbf{ then } c[k] := a[i]; k := k + 1; B := false; i := i + 1 \textbf{ end } \{I_1\} \\ \textbf{od } \{I_1 \wedge i \geq M\} \end{array}$$

Thread 2

$$\begin{array}{l} \{inv : I_2\} \{bd : t_2\} \\ \textbf{while } i < M \textbf{ do } \{I_2 \wedge j < M\} \\ \quad \textbf{await } \neg B \textbf{ then } c[k] := a[i]; k := k + 1; B := false; i := i + 1 \textbf{ end } \{I_2\} \\ \textbf{od } \{I_2 \wedge i \geq M\} \end{array}$$

Main

$$\{true\} \ S' \ \{\forall l \in \{0, \dots, 2M - 1\}. (even(l) \rightarrow c[l] = a[l/2]) \wedge (odd(l) \rightarrow c[l] = b[(l - 1)/2])\}$$

3: Prove Total Correctness

Thread 1

Invariant: $I_1 = 0 \leq i \leq M \wedge k = i + j \wedge I$

Termination Function: $t_1 = M - i$

Thread 2

Invariant: $I_2 = 0 \leq j \leq M \wedge k = i + j \wedge I$

Termination Function: $t_2 = M - j$

$$I = (B \rightarrow (\text{even}(k) \wedge i = j)) \wedge (\neg B \rightarrow (\text{odd}(k) \wedge i = j + 1)) \wedge \\ \forall l \in \{0, \dots, k - 1\}. ((\text{even}(l) \rightarrow c[l] = a[l/2]) \wedge (\text{odd}(l) \rightarrow c[l] = b[(l - 1)/2]))$$

3: Prove Total Correctness

Thread 1

Invariant: $I_1 = 0 \leq i \leq M \wedge k = i + j \wedge I$

Termination Function: $t_1 = M - i$

Thread 2

Invariant: $I_2 = 0 \leq j \leq M \wedge k = i + j \wedge I$

Termination Function: $t_2 = M - j$

$$I = (B \rightarrow (\text{even}(k) \wedge i = j)) \wedge (\neg B \rightarrow (\text{odd}(k) \wedge i = j + 1)) \wedge \\ \forall l \in \{0, \dots, k - 1\}. ((\text{even}(l) \rightarrow c[l] = a[l/2]) \wedge (\text{odd}(l) \rightarrow c[l] = b[(l - 1)/2]))$$

Derive proof by pattern matching of the rules

RULE 26: ATOMIC REGION

$$\frac{\{p\} S \{q\}}{\{p\} \langle S \rangle \{q\}}$$

RULE 3: COMPOSITION

$$\frac{\{p\} S_1 \{r\}, \{r\} S_2 \{q\}}{\{p\} S_1; S_2 \{q\}}$$

RULE 7: LOOP II

$$\frac{\begin{array}{l} \{p \wedge B\} S \{p\}, \\ \{p \wedge B \wedge t = z\} S \{t < z\}, \\ p \rightarrow t \geq 0 \end{array}}{\{p\} \text{ while } B \text{ do } S \text{ od } \{p \wedge \neg B\}}$$

RULE 27: PARALLELISM WITH SHARED VARIABLES

$$\frac{\begin{array}{l} \text{The standard proof outlines } \{p_i\} S_i^* \{q_i\}, \\ i \in \{1, \dots, n\}, \text{ are interference free} \end{array}}{\{\bigwedge_{i=1}^n p_i\} [S_1 \parallel \dots \parallel S_n] \{\bigwedge_{i=1}^n q_i\}}$$

RULE 28: SYNCHRONIZATION

$$\frac{\{p \wedge B\} S \{q\}}{\{p\} \text{ await } B \text{ then } S \text{ end } \{q\}}$$

4: Prove Deadlock Free

Find potential deadlock points regions

$$\begin{aligned} &(\text{await } B \dots, E) \quad (0 \leq i \leq M \wedge (\neg B \rightarrow (\text{odd}(k) \wedge i = j + 1)) \wedge \dots \wedge \neg B, j = M \wedge \dots) \\ &(E, \text{await } \neg B \dots) \quad (i = M \wedge \dots, j < M \wedge (B \rightarrow (\text{even}(k) \wedge i = j)) \wedge \dots \wedge B) \\ &(\text{await } B \dots, \text{await } \neg B \dots) \quad (\neg B \wedge \dots, B \wedge \dots) \end{aligned}$$

Derive proof by pattern matching of the rule

RULE 29: PARALLELISM WITH DEADLOCK FREEDOM

- (1) The standard proof outlines $\{p_i\} S_i^* \{q_i\}, i \in \{1, \dots, n\}$ for weak total correctness are interference free,
- (2) For every potential deadlock $(R_1, \dots, R_n)$ of $[S_1 \parallel \dots \parallel S_n]$ the corresponding tuple of assertions $(r_1, \dots, r_n)$ satisfies $\neg \bigwedge_{i=1}^n r_i$.

$$\{\bigwedge_{i=1}^n p_i\} [S_1 \parallel \dots \parallel S_n] \{\bigwedge_{i=1}^n q_i\}$$

5: Translate to a real language

Thread 1

```
20 void S1(){
21     while(i < M){
22         { // await B
23             std::unique_lock<std::mutex> lk(mx);
24             cv.wait(lk, [](){return B;});
25             c[k] = a[i];
26             k++;
27             B = false;
28             cv.notify_all();
29         }
30         i++;
31     }
32 }
```

Thread 2

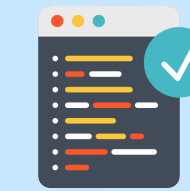
```
34 void S2(){
35     while(j < M){
36         { // await !B
37             std::unique_lock<std::mutex> lk(mx);
38             cv.wait(lk, [](){return !B;});
39             c[k] = b[j];
40             k++;
41             B = true;
42             cv.notify_all();
43         }
44         j++;
45     }
46 }
```

Main

```
48 int main(){
49     // S'
50     k = 0;
51     B = true;
52     i = 0; j = 0;
53
54     // [S1 | S2]
55     auto thread1 = std::async(std::launch::async, S1);
56     auto thread2 = std::async(std::launch::async, S2);
57     thread1.get();
58     thread2.get();
59 }
```


Yes... Good in theory...

Yes... Good in theory...



Yes... Good in theory...

What can we do ?

Use Proof Assistant software like ROCQ

- C language support
- Useful to prove math functions

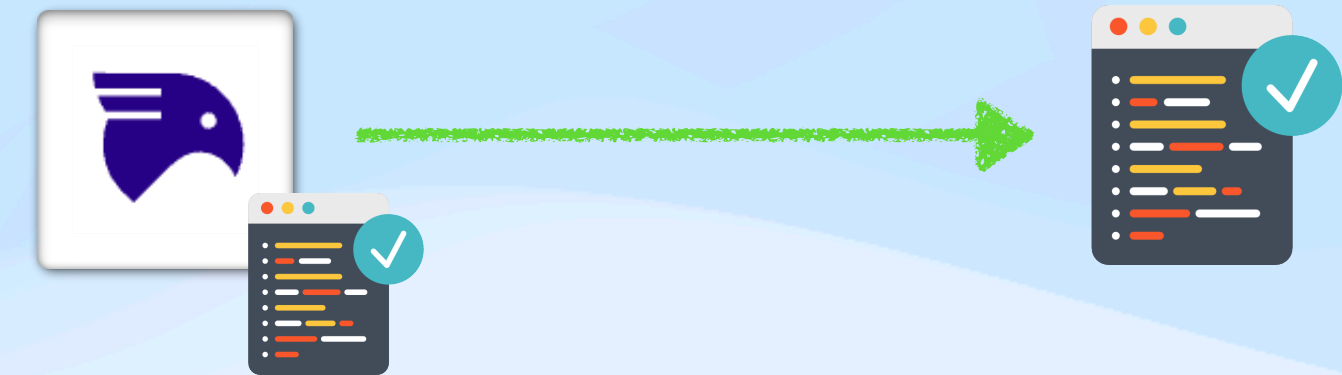
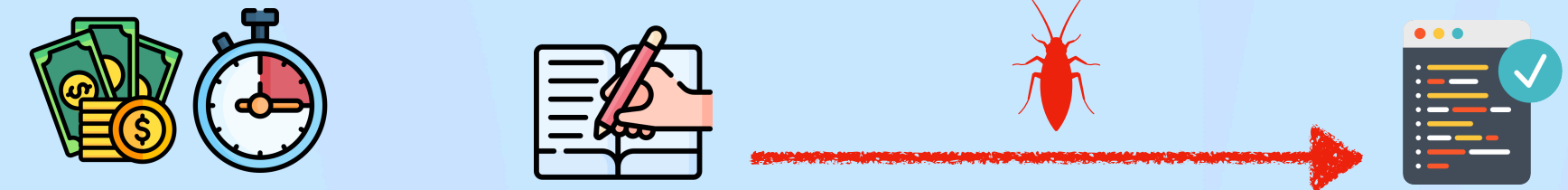


Yes... Good in theory...

What can we do ?

Use Proof Assistant software like ROCQ

- C language support
- Useful to prove math functions

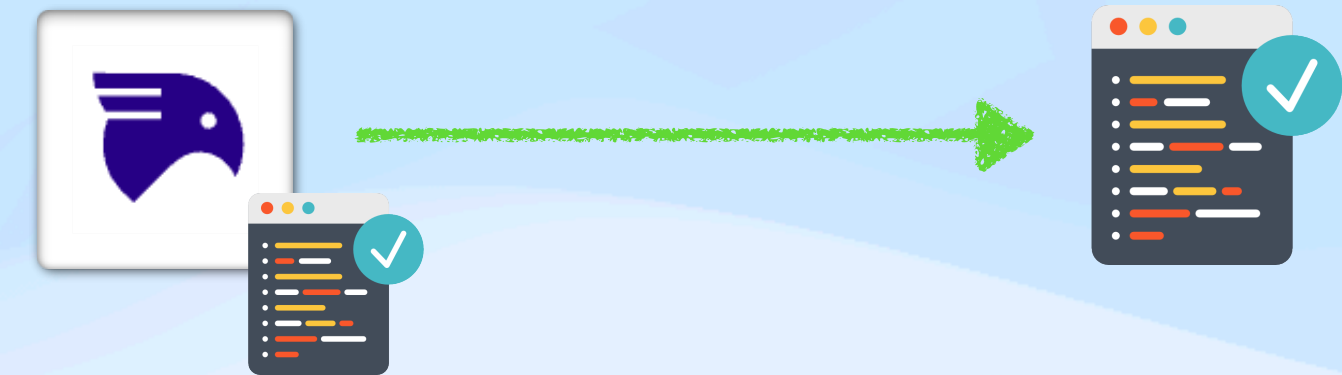
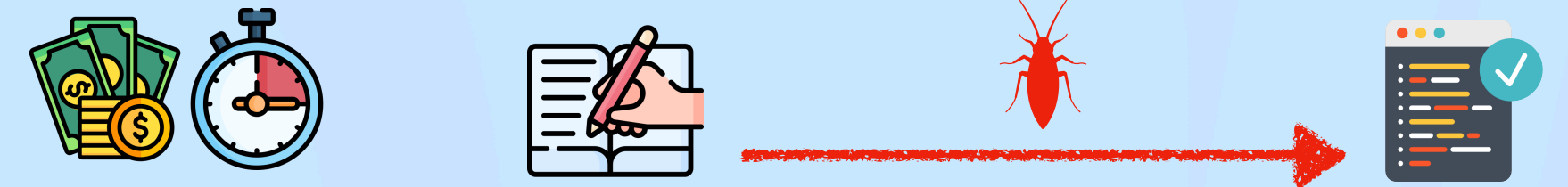


Yes... Good in theory...

What can we do ?

Use Proof Assistant software like ROCQ

- C language support
- Useful to prove math functions



Problems

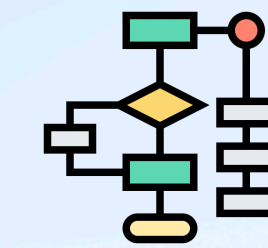
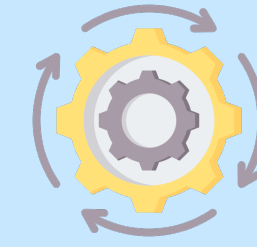
- Very difficult to learn and use
- Need language translation*

Writing a proof for everything is unfeasible

Writing a proof for everything is unfeasible

Lets prove what's necessary

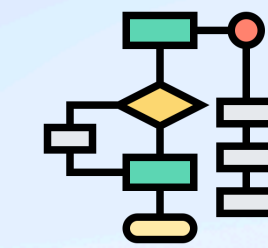
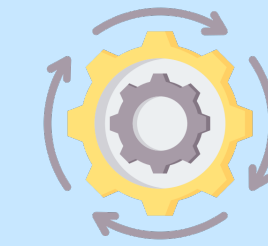
- System core
- Math functions
- Frequently used algorithms



Writing a proof for everything is unfeasible

Lets prove what's necessary

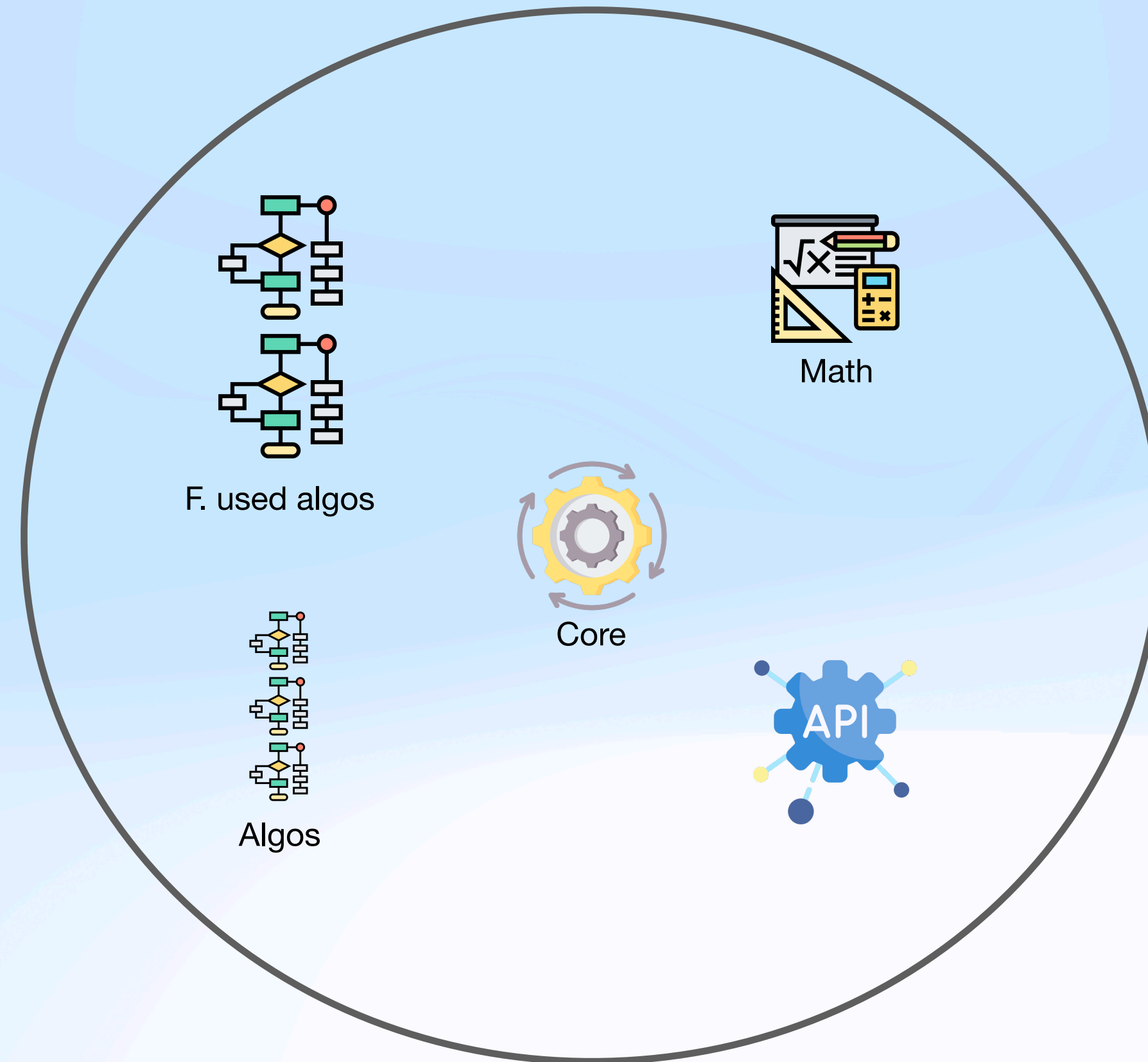
- System core
- Math functions
- Frequently used algorithms



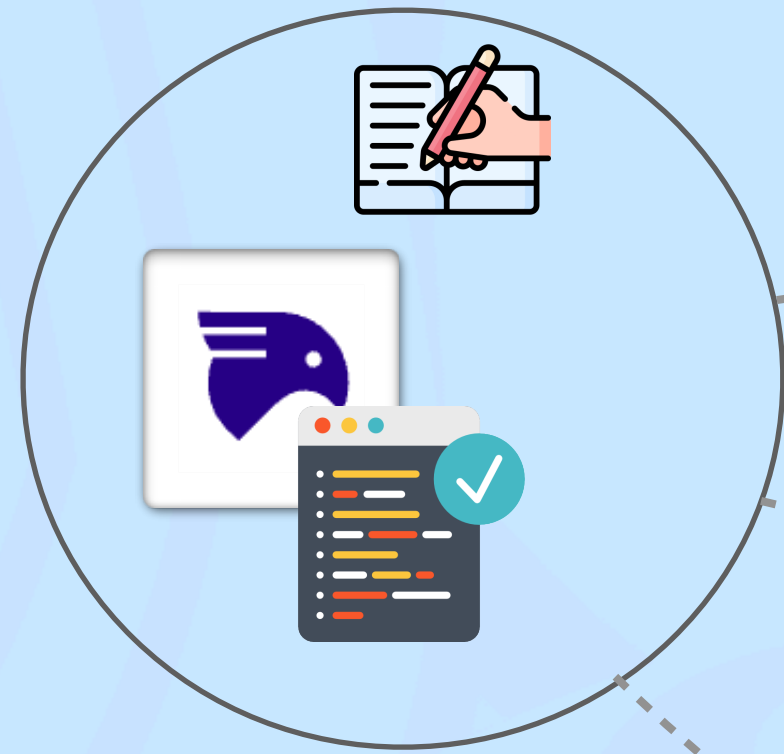
Use tests for APIs and unproved parts

Simulate complex interactions

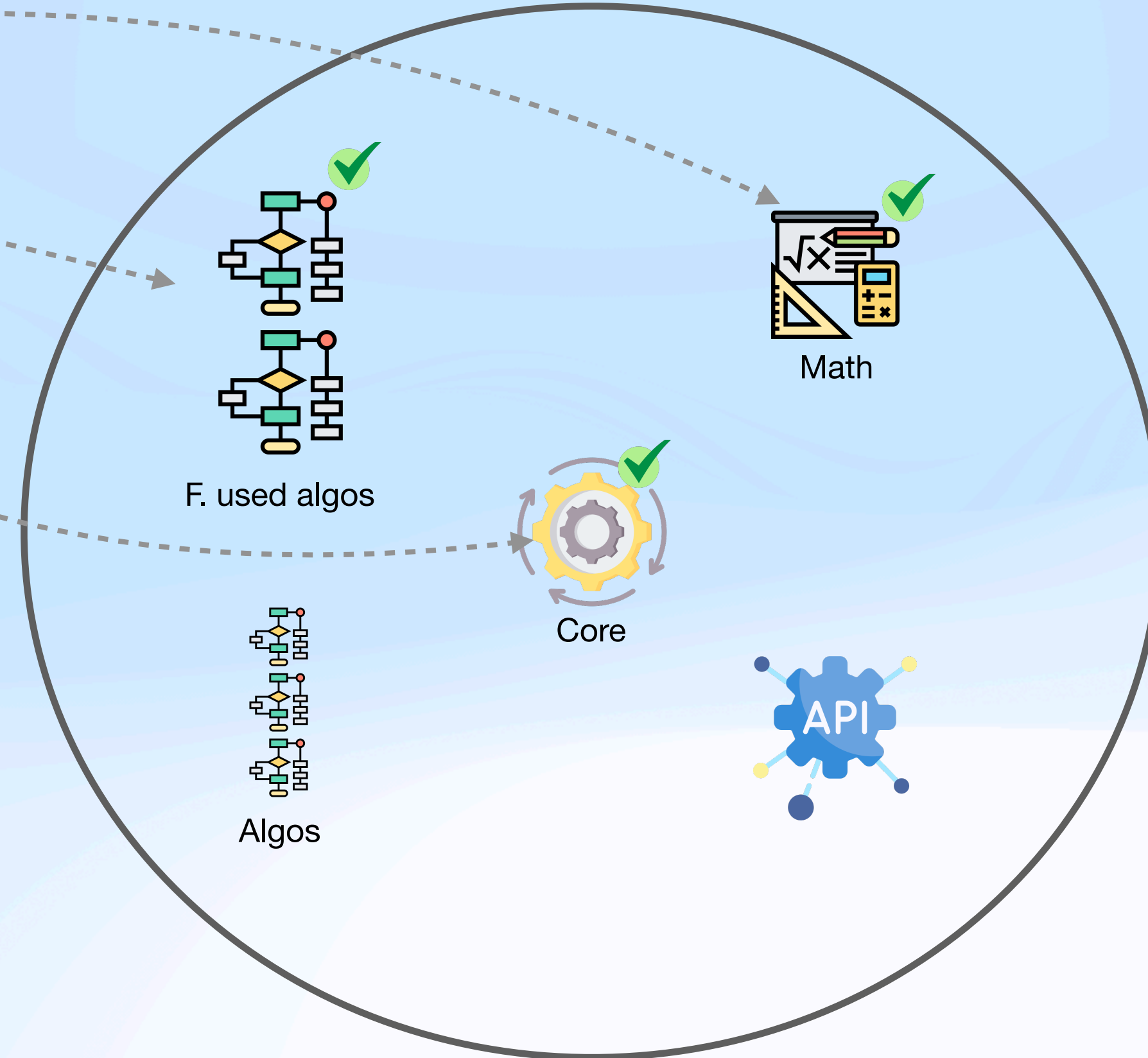
System



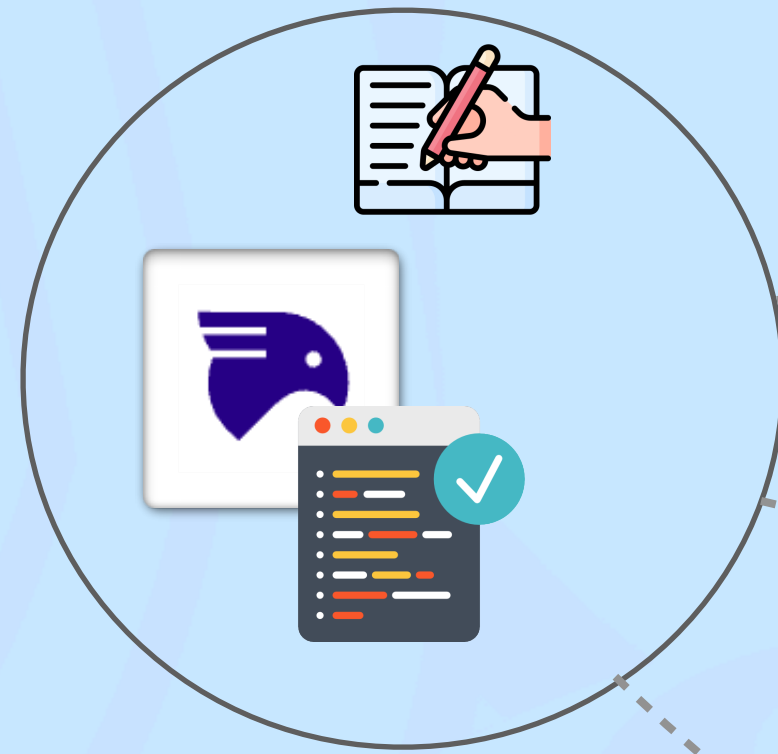
Formal Methods



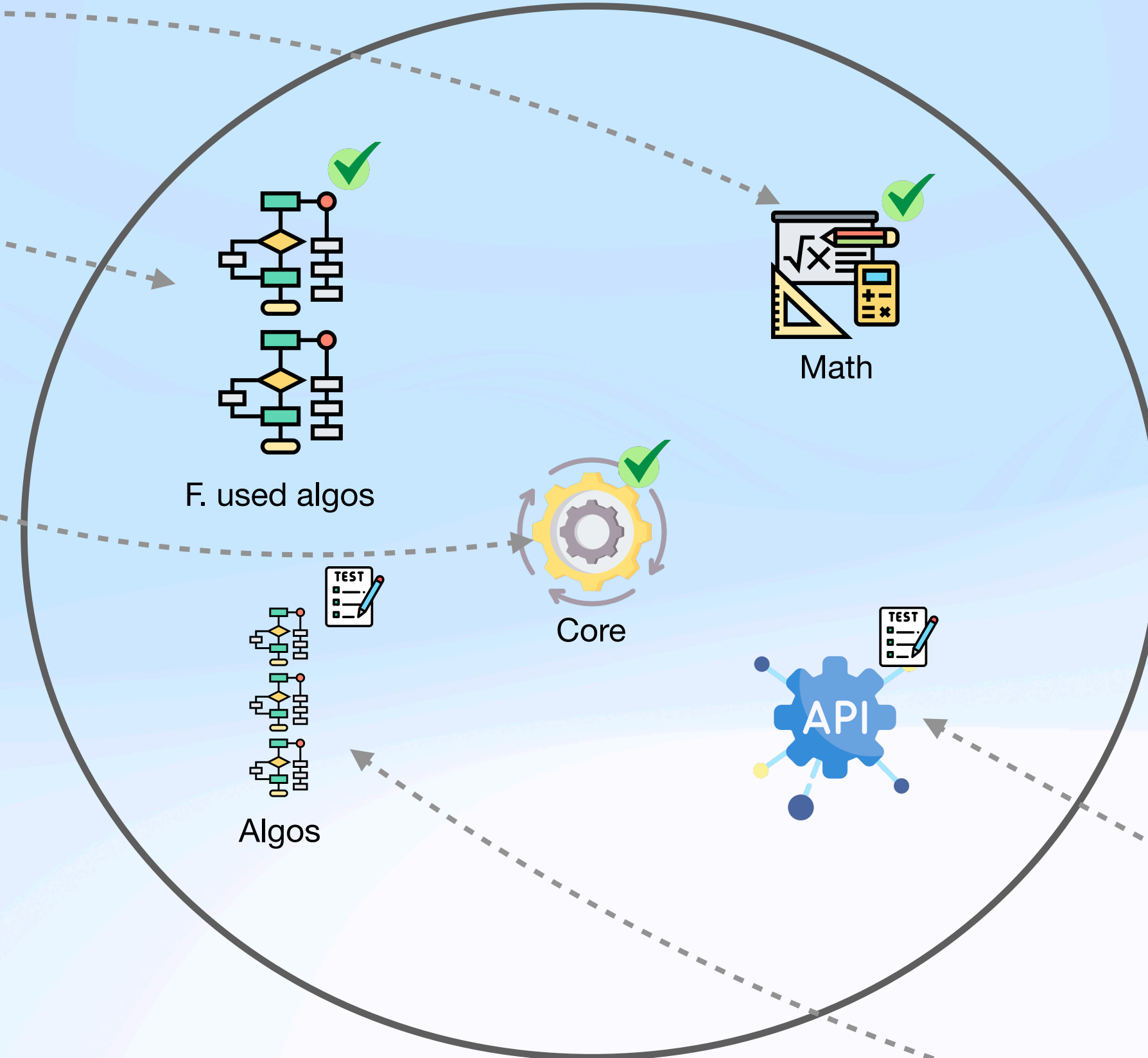
System



Formal Methods



System



F. used algos

Math

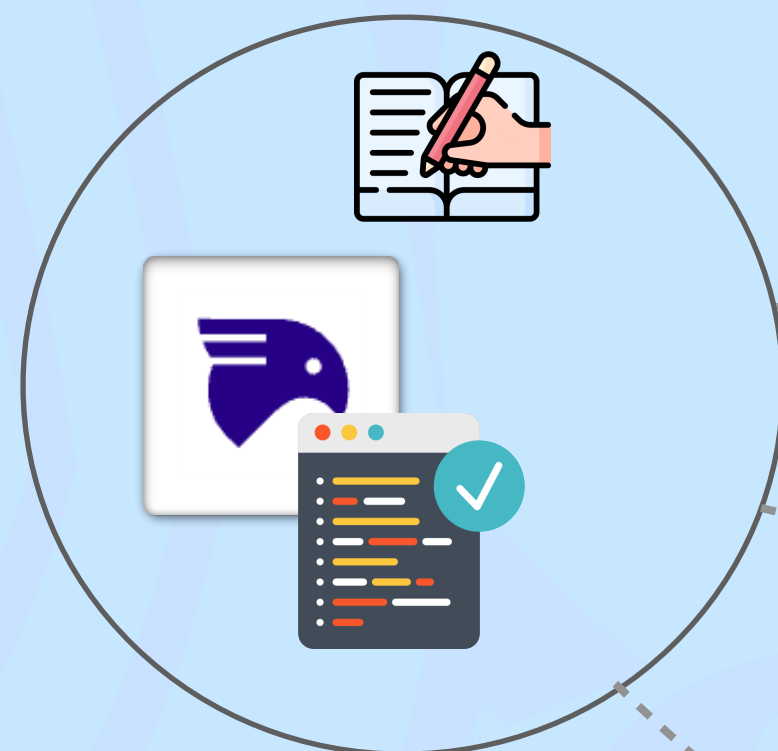
Core

API

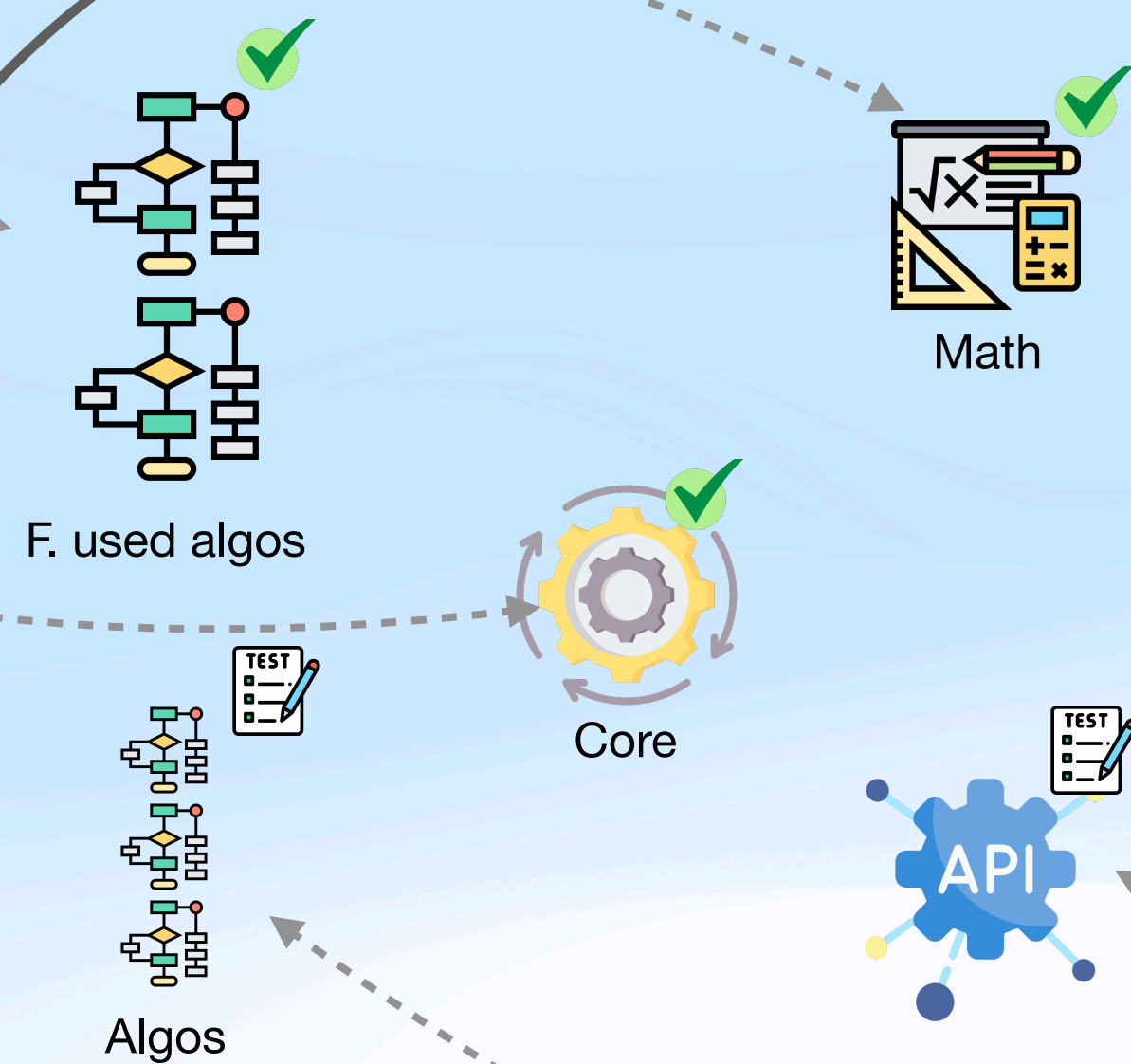
Algos



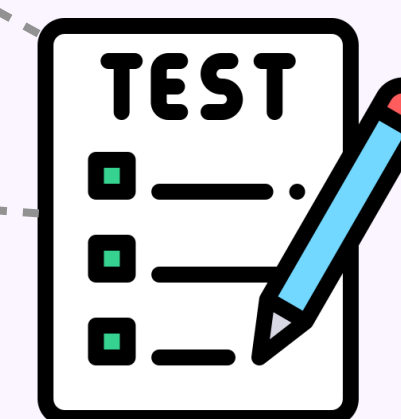
Formal Methods



System



Complex interactions / Training / RL



Thank you

Questions ?

Interesting links:

https://en.wikipedia.org/wiki/CAP_theorem

<https://rocq-prover.org/>

<https://softwarefoundations.cis.upenn.edu/>

<https://ir.cwi.nl/pub/14569/14569A.pdf>